by Scala from the context, the type annotations on the function's arguments may be elided, for example, `(x,y) => x < y`. We'll see an example of this in the next section, and lots more examples throughout this book.

Functions as values in Scala

When we define a function literal, what is actually being defined in Scala is an object with a method called `apply`. Scala has a special rule for this method name, so that objects that have an `apply` method can be called as if they were themselves methods. When we define a function literal like `(a, b) => a < b`, this is really syntactic sugar for object creation:

```
val lessThan = new Function2[Int, Int, Boolean] {
  def apply(a: Int, b: Int) = a < b
}
```

`lessThan` has type `Function2[Int,Int,Boolean]`, which is usually written `(Int,Int) => Boolean`. Note that the `Function2` interface (known in Scala as a *trait*) has an `apply` method. And when we call the `lessThan` function with `lessThan(10, 20)`, it's really syntactic sugar for calling its `apply` method:

```
scala> val b = lessThan.apply(10, 20)
b: Boolean = true
```

`Function2` is just an ordinary trait (an interface) provided by the standard Scala library (API docs link: http://mng.bz/qFMr) to represent function objects that take two arguments. Also provided are `Function1`, `Function3`, and others, taking a number of arguments indicated by the name. Because functions are just ordinary Scala objects, we say that they're *first-class* values. We'll often use "function" to refer to either such a first-class function or a method, depending on context.

2.6 Following types to implementations

As you might have seen when writing `isSorted`, the universe of possible implementations is significantly reduced when implementing a polymorphic function. If a function is polymorphic in some type `A`, the only operations that can be performed on that `A` are those passed into the function as arguments (or that can be defined in terms of these given operations).[6] In some cases, you'll find that the universe of possibilities for a given polymorphic type is constrained such that only one implementation is possible!

Let's look at an example of a function signature that can only be implemented in one way. It's a higher-order function for performing what's called *partial application*. This function, `partial1`, takes a value and a function of two arguments, and returns a function of one argument as its result. The name comes from the fact that the function is being applied to some but not all of the arguments it requires:

```
def partial1[A,B,C](a: A, f: (A,B) => C): B => C
```

[6] Technically, all values in Scala can be compared for equality (using `==`), and turned into strings with `toString` and integers with `hashCode`. But this is something of a wart inherited from Java.

The `partial1` function has three type parameters: `A`, `B`, and `C`. It then takes two arguments. The argument `f` is itself a function that takes two arguments of types `A` and `B`, respectively, and returns a value of type `C`. The value returned by `partial1` will also be a function, of type `B => C`.

How would we go about implementing this higher-order function? It turns out that there's only one implementation that compiles, and it follows logically from the type signature. It's like a fun little logic puzzle.[7]

Let's start by looking at the type of thing that we have to return. The return type of `partial1` is `B => C`, so we know that we have to return a function of that type. We can just begin writing a function literal that takes an argument of type `B`:

```
def partial1[A,B,C](a: A, f: (A,B) => C): B => C =
  (b: B) => ???
```

This can be weird at first if you're not used to writing anonymous functions. Where did that `B` come from? Well, we've just written, "Return a function that takes a value `b` of type `B`." On the right-hand-side of the `=>` arrow (where the question marks are now) comes the body of that anonymous function. We're free to refer to the value `b` in there for the same reason that we're allowed to refer to the value `a` in the body of `partial1`.[8]

Let's keep going. Now that we've asked for a value of type `B`, what do we want to return from our anonymous function? The type signature says that it has to be a value of type `C`. And there's only one way to get such a value. According to the signature, `C` is the return type of the function `f`. So the only way to get that `C` is to pass an `A` and a `B` to `f`. That's easy:

```
def partial1[A,B,C](a: A, f: (A,B) => C): B => C =
  (b: B) => f(a, b)
```

And we're done! The result is a higher-order function that takes a function of two arguments and partially applies it. That is, if we have an `A` and a function that needs both `A` and `B` to produce `C`, we can get a function that just needs `B` to produce `C` (since we already have the `A`). It's like saying, "If I can give you a carrot for an apple and a banana, and you already gave me an apple, you just have to give me a banana and I'll give you a carrot."

Note that the type annotation on `b` isn't needed here. Since we told Scala the return type would be `B => C`, Scala knows the type of `b` from the context and we could just write `b => f(a,b)` as the implementation. Generally speaking, we'll omit the type annotation on a function literal if it can be inferred by Scala.

---

[7] Even though it's a fun puzzle, this isn't a purely academic exercise. Functional programming in practice involves a lot of fitting building blocks together in the only way that makes sense. The purpose of this exercise is to get practice using higher-order functions, and using Scala's type system to guide your programming.

[8] Within the body of this inner function, the outer `a` is still in scope. We sometimes say that the inner function *closes over* its environment, which includes `a`.

EXERCISE 2.3

Let's look at another example, *currying*,[9] which converts a function `f` of two arguments into a function of one argument that partially applies `f`. Here again there's only one implementation that compiles. Write this implementation.

```
def curry[A,B,C](f: (A, B) => C): A => (B => C)
```

EXERCISE 2.4

Implement `uncurry`, which reverses the transformation of `curry`. Note that since `=>` associates to the right, `A => (B => C)` can be written as `A => B => C`.

```
def uncurry[A,B,C](f: A => B => C): (A, B) => C
```

Let's look at a final example, *function composition*, which feeds the output of one function to the input of another function. Again, the implementation of this function is fully determined by its type signature.

EXERCISE 2.5

Implement the higher-order function that composes two functions.

```
def compose[A,B,C](f: B => C, g: A => B): A => C
```

This is such a common thing to want to do that Scala's standard library provides `compose` as a method on `Function1` (the interface for functions that take one argument). To compose two functions `f` and `g`, we simply say `f compose g`.[10] It also provides an `andThen` method. `f andThen g` is the same as `g compose f`:

```
scala> val f = (x: Double) => math.Pi / 2 - x
f: Double => Double = <function1>

scala> val cos = f andThen math.sin
cos: Double => Double = <function1>
```

It's all well and good to puzzle together little one-liners like this, but what about programming with a large real-world code base? In functional programming, it turns out to be exactly the same. Higher-order functions like `compose` don't care whether

[9] This is named after the mathematician Haskell Curry, who discovered the principle. It was independently discovered earlier by Moses Schoenfinkel, but *Schoenfinkelization* didn't catch on.

[10] Solving the `compose` exercise by using this library function is considered cheating.